

METRISCAN AI

SYSTEM ARCHITECTURE SPECIFICATION

Technical Architecture for the SIH Problem Statement 26034

Product: AI-Assisted Legal Metrology Compliance Inspection Platform

Organization: Ministry of Consumer Affairs, Food & Public Distribution

Department: Department of Consumer Affairs (DoCA)

Architecture Version: 1.0 — SIH MVP

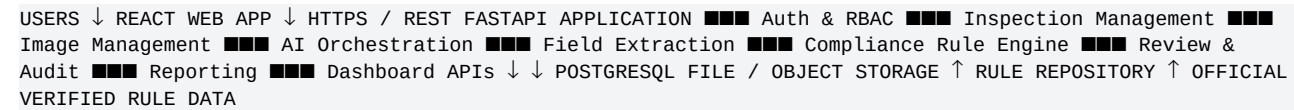
Architecture philosophy: Keep the MVP modular and explainable. AI extracts information; a deterministic, versioned rule engine evaluates compliance; humans review uncertain cases; every finding is backed by evidence.

1. Architecture Goals

- Provide a complete path from package image to compliance report.
- Separate AI extraction from legal/compliance decision logic.
- Make every result traceable to an image, extracted field and rule version.
- Support human review and correction without losing the original AI output.
- Allow rules to change without rewriting application code.
- Keep the architecture simple enough for a student SIH team to implement.
- Provide a clean upgrade path from a modular monolith to distributed services later.

2. Recommended Architecture Style

Recommended for SIH: Modular Monolith + Dedicated AI Module + PostgreSQL + Object/File Storage.

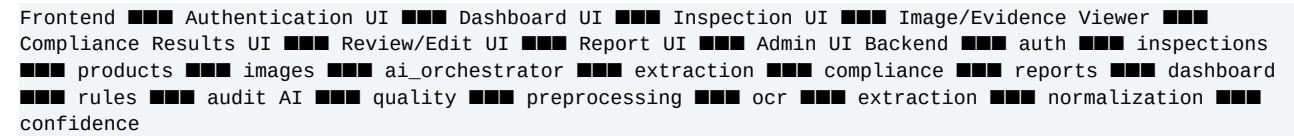


Do not begin with many microservices. The team can separate OCR, rules or reporting into services later if load or deployment requirements justify it.

3. Logical Architecture Layers

Layer	Responsibilities	Main Technology
Presentation	Login, inspection workflow, image upload, results, evidence, reports, dashboard	React + Vite + Tailwind
API / Application	Authentication, authorization, orchestration, validation, CRUD, workflow	Python + FastAPI
AI / CV	Quality checks, preprocessing, OCR, field extraction, confidence	OpenCV + PaddleOCR + Python
Domain / Rules	Applicability, declaration checks, aggregation, severity	Python rule engine + versioned rule data
Persistence	Users, inspections, OCR, fields, rules, violations, reports, audit logs	PostgreSQL
File Storage	Original/processed images and generated reports	Local object storage for MVP; S3-compatible later
Infrastructure	Containers, environment variables, logging, backups	Docker + GitHub + deployment platform

4. Component Architecture



5. End-to-End Data Flow

1. Officer logs in 2. Creates inspection 3. Uploads front/back/side package images 4. Backend stores originals 5. AI pipeline checks quality 6. Image preprocessing improves the image 7. OCR produces text + bounding boxes + confidence 8. Extraction converts OCR into structured declarations 9. Normalization standardizes units/values 10. Rule engine selects applicable rule version 11. Validators evaluate each requirement 12. Result aggregator creates PASS / FAIL / REVIEW / NOT_APPLICABLE 13. Evidence links each finding to image region + extracted value + rule 14. Officer/reviewer corrects uncertain values if required 15. Rules are re-evaluated after corrections 16. Inspection is finalized 17. PDF report is generated and stored 18. Dashboard/history update automatically

6. Main Sequence Diagram — Inspection Analysis

Officer → Frontend: Upload images Frontend → API: POST /inspections/{id}/images API → Storage: Save originals API → AI Orchestrator: Analyze inspection AI → Quality Service: Check image AI → Preprocessor: Enhance / skew / resize AI → OCR: Extract text + boxes + confidence OCR → Extraction: Raw OCR result Extraction → Normalizer: Structured fields Normalizer → Compliance Engine: Fields + context Compliance Engine → Rule Repository: Applicable rule versions Rule Engine → Validators: Evaluate requirements Validators → Compliance Engine: Rule results Compliance Engine → API: Findings + evidence API → Frontend: Analysis result Frontend → Officer: PASS / FAIL / REVIEW

7. AI / Computer Vision Architecture

IMAGE INGESTION ↓ QUALITY CHECK ■■■ blur detection ■■■ brightness/contrast check ■■■ resolution check ■■■ orientation check ↓ PREPROCESSING ■■■ resize ■■■ crop/region detection ■■■ skew ■■■ perspective correction ■■■ denoise ■■■ contrast/sharpening ↓ OCR ↓ TEXT + BOUNDING BOX + CONFIDENCE ↓ FIELD EXTRACTION ■■■ MRP ■■■ quantity ■■■ dates ■■■ manufacturer/packer/importer ■■■ product name ■■■ consumer-care details ↓ NORMALIZATION ↓ CONFIDENCE / REVIEW GATE

Important: an LLM may optionally help with ambiguous text classification or semantic mapping, but its output must not become an unsupported legal conclusion. The deterministic rule engine remains the authority for machine-checkable rules.

8. Field Extraction Architecture

Stage	Example
Raw OCR	"MAXIMUM RETAIL PRICE Rs. 120.00 incl. of all taxes"
Candidate detection	MRP candidate = 120.00
Normalization	currency=INR, value=120.00
Evidence	image_id + bounding_box
Confidence	0.97
Validation	MRP field present and machine-checkable conditions evaluated
Result	PASS / FAIL / REVIEW

9. Compliance Rule Engine Architecture

RULE REPOSITORY ↓ APPLICABILITY ENGINE ↓ REQUIREMENT RESOLVER ↓ FIELD VALIDATORS ■■■ presence validator ■■■ format validator ■■■ value validator ■■■ consistency validator ■■■ readability/confidence gate ↓ SEVERITY CLASSIFIER ↓ RESULT AGGREGATOR ↓ INSPECTION STATUS

Rule execution should be deterministic, testable and version-aware.

10. Rule Versioning Model

RULE ■■■ rule_id ■■■ code ■■■ title ■■■ requirement_type ■■■ applicability_conditions ■■■ validation_logic ■■■ severity ■■■ source_reference RULE_VERSION ■■■ version_id ■■■ rule_id ■■■ version ■■■ effective_from ■■■ effective_to ■■■ definition_snapshot ■■■ active

When an inspection is finalized, its rule results must point to the exact rule version used. This ensures an old inspection can be reproduced even after the rules change.

11. Database Architecture

USERS < INSPECTIONS > PRODUCTS < IMAGES < OCR_RESULTS < EXTRACTED_FIELDS < RULE_RESULTS > RULE_VERSIONS > RULES < VIOLATIONS < REPORTS < AUDIT_LOGS

Core tables

Table	Important Columns
users	id, name, email, password_hash, role_id, status, created_at
roles	id, name
products	id, brand, product_name, category, metadata
inspections	id, product_id, officer_id, location, status, started_at, finalized_at
images	id, inspection_id, file_path, image_type, quality_score, created_at
ocr_results	id, image_id, text, confidence, bounding_boxes, engine_version
extracted_fields	id, inspection_id, field_name, raw_value, normalized_value, unit, confidence, evidence_ref, verification_status
rules	id, code, title, category, requirement_type, severity
rule_versions	id, rule_id, version, effective_from, effective_to, definition, source_reference
rule_results	id, inspection_id, rule_version_id, result, reason, evidence_ref
violations	id, inspection_id, rule_result_id, severity, description, status
reports	id, inspection_id, version, file_path, generated_by, created_at
audit_logs	id, user_id, entity_type, entity_id, action, old_value, new_value, created_at

12. API Architecture

/api/auth POST /login GET /me /api/inspections POST / GET / GET /{id} PATCH /{id} POST /{id}/finalize /api/images POST /inspections/{id}/images GET /{id} /api/analysis POST /inspections/{id}/analyze GET /inspections/{id}/results /api/review PATCH /fields/{id} POST /inspections/{id}/rerun /api/rules GET / POST / GET /{id} POST /{id}/versions /api/reports POST /inspections/{id}/generate GET /{id} /api/dashboard GET /summary GET /trends GET /violations

API rules

- All protected endpoints require authentication.
- Authorization is checked server-side.
- Use consistent JSON error responses.
- Validate file uploads and request bodies.
- Return stable IDs for inspections, images, rules and reports.
- Use pagination for history/search endpoints.

13. Frontend Architecture

src/ app/ pages/ Login Dashboard NewInspection InspectionDetails AnalysisResults EvidenceViewer ReviewInspection History Reports Admin components/ ImageUploader FieldTable ComplianceBadge EvidencePanel ViolationCard Charts services/api.js hooks/ utils/ styles/

14. Backend Architecture & Folder Structure

backend/ app/ main.py core/ config.py security.py database.py api/ auth.py inspections.py analysis.py review.py rules.py reports.py dashboard.py models/ schemas/ services/ inspection_service.py ai_service.py extraction_service.py compliance_service.py report_service.py audit_service.py rules/ tests/

15. AI Module Structure

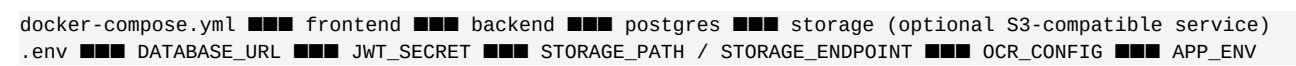
```
ai/ ■■■ preprocessing/ ■ ■■■ quality.py ■ ■■■ orientation.py ■ ■■■ deskew.py ■ ■■■ enhancement.py ■■■  
ocr/ ■ ■■■ engine.py ■ ■■■ postprocess.py ■■■ extraction/ ■ ■■■ field_detector.py ■ ■■■ mrp.py ■ ■■■  
quantity.py ■ ■■■ dates.py ■ ■■■ manufacturer.py ■ ■■■ consumer_care.py ■■■ normalization/ ■ ■■■  
units.py ■ ■■■ text.py ■■■ confidence/ ■ ■■■ scorer.py ■■■ tests/
```

16. Deployment Architecture



For the SIH demo, all components can run with Docker Compose on one machine/server. A later production architecture can move PostgreSQL and object storage to managed services and scale API/AI workers independently.

17. Docker / Environment Layout



18. Security Architecture

- HTTPS for deployed environments.
- Password hashing with a strong password-hashing algorithm.
- Short-lived access tokens and secure token handling.
- Role-based access checks in backend dependencies/middleware.
- Input validation and upload restrictions.
- Parameterized database operations through ORM.
- Audit trail for sensitive changes.
- Environment secrets never committed to Git.
- Separate development/test data from real inspection data.
- Backups and retention policy for production deployment.

19. Logging, Monitoring & Audit

Type	Record
Application logs	Request ID, endpoint, duration, success/failure
AI logs	Image ID, pipeline stage, model/OCR version, processing time, confidence summary
Compliance logs	Inspection ID, rule version, result, reason
Audit logs	User, action, entity, previous/new value, timestamp
System metrics	CPU, memory, request rate, errors, analysis duration

20. Failure & Recovery Architecture

Failure	Recovery
Image upload fails	Keep inspection draft; allow retry.
OCR fails	Mark image analysis failed; allow retry or manual entry.
AI service unavailable	Inspection remains pending; no false compliance result.
Rule configuration missing	Return REVIEW/configuration error; block finalization.
Report generation fails	Keep finalized data; allow report regeneration.
Database temporary failure	Return recoverable error; transaction rollback.
Low OCR confidence	Route to REVIEW instead of inventing a value.

21. Testing Architecture

- **Unit tests:** field parsers, normalizers, rule validators and status aggregation.

- **Integration tests:** FastAPI + PostgreSQL + rule repository.
- **AI evaluation tests:** OCR and extraction against manually verified images.
- **End-to-end tests:** image upload → analysis → result → review → report.
- **Security tests:** unauthorized API access, role restrictions and upload validation.
- **Regression suite:** maintain a golden set of representative package images and expected structured results.

22. GitHub / CI Pipeline

Developer branch → Pull Request → lint/format → unit tests → API tests → build Docker images → integration tests → merge → deploy

Recommended repository layout: metriscan-ai/  frontend/  backend/  ai/  rules/  docs/  dataset/ # only permitted/non-sensitive development data  tests/  docker-compose.yml  README.md

23. Key Architecture Decisions

Decision	Why
React + FastAPI	Fast development, clear frontend/backend separation and Python AI compatibility.
PostgreSQL	Strong relational model for inspections, rules, versions and audit data.
Modular monolith	Reduces SIH implementation complexity.
OCR + deterministic extraction	Transparent and testable for structured declarations.
Rule engine separate from OCR	Prevents AI extraction errors from becoming hidden legal logic.
Evidence-first storage	Makes findings explainable and reviewable.
Rule versioning	Makes historical decisions reproducible.
Human-in-the-loop	Handles OCR ambiguity and legal uncertainty safely.
Docker	Reproducible local/demo deployment.

24. Future Production Scaling

MVP: Single server → Docker Compose → FastAPI + PostgreSQL + local/object storage. Later: Load Balancer → Multiple API instances → Job Queue → Dedicated AI workers → PostgreSQL HA → Object Storage → Monitoring

Only introduce a queue and separate AI workers when analysis volume requires asynchronous processing. The logical boundaries in the MVP are designed so this migration can happen without redesigning the product.

25. Complete System Architecture Summary

USER ↓ REACT FRONTEND ↓ FASTAPI API + AUTH/RBAC ↓ INSPECTION SERVICE ↓ IMAGE STORAGE + AI ORCHESTRATOR ↓ QUALITY → PREPROCESSING → OCR → EXTRACTION → NORMALIZATION → CONFIDENCE ↓ VERSIONED COMPLIANCE RULE ENGINE ↓ PASS / FAIL / REVIEW / NOT_APPLICABLE ↓ EVIDENCE + AUDIT + HUMAN REVIEW ↓ FINALIZED INSPECTION ↓ PDF REPORT + HISTORY + DASHBOARD

26. Build Order for the Team

- **Step 1:** Create GitHub repository and project structure.
- **Step 2:** Build database schema and FastAPI skeleton.
- **Step 3:** Build the Python image → OCR → extraction prototype.
- **Step 4:** Build the rule repository and deterministic compliance engine.
- **Step 5:** Connect AI pipeline to FastAPI.
- **Step 6:** Build React inspection workflow.
- **Step 7:** Build evidence/review screen.
- **Step 8:** Build PDF reporting and inspection history.
- **Step 9:** Build dashboard/admin screens.
- **Step 10:** Integrate, test, Dockerize and prepare the SIH demo.

Architecture completion criterion: A judge should be able to upload a prepared package image, watch the system extract declarations, inspect the evidence behind a finding, see the applicable rule/result, correct a low-confidence field, finalize the inspection and receive a professional report — all from one coherent application.